

Proyecto Integrador: Diseño de Arquitectura Híbrida para Ecommify

Asignatura: Diseño y Optimización de Bases de Datos

Actividad: Guía de Actividad Evaluativa Unidad 2

Fecha: Semana del 4 al 10 de Mayo

Integrantes del Equipo:
Juan Daniel Valderrama Pérez
Jorge Esteban Triviño Correa
Javier Andres Baron Fontanilla

Docente: Miguel Alfonso Varela Fonseca

Universidad de la Sabana
Facultad de Ingeniería

Tabla de Contenido

Enfoque Arquitectónico Seleccionado	4
Abstract	4
Análisis de Requisitos	4
• Contexto de Ecomimfy	4
• Requisitos funcionales	5
• Requisitos no funcionales	5
• Identificación de entidades del dominio	6
◦ PostgreSQL - Transaccional	6
• MongoDB - Documental	6
• Tabla de volúmenes de datos esperados	6
• Justificación arquitectónica	7
Diseño Conceptual	8
• Diagrama ER completo de alta calidad - Sin Normalizar	8
Figura 1. Diagrama ER Sin Normalizar del Dataset Principal Ollist.	8
• Descripción detallada de cada entidad y relación	8
• Descripción de Relaciones	9
• Restricciones de negocio identificadas	9
◦ Integridad Transaccional - Pedidos y Pagos	9
◦ Logística y Vendedores	10
◦ Calidad y Experiencia - Reviews	10
Diseño lógico - Módulo PostgreSQL	10
• Esquema relacional normalizado	10
• Tabla geolocation	10
• Tabla customers	11
• Tabla sellers	11
• Tabla products	11
• Tabla orders	11
• Tabla order_items	12
• Tabla payments	12
• Tabla promotions	12
Figura 2. Diagrama ER Normalizado (3FN) para el Diseño Propuesto del Módulo PostgreSQL.	13
• Aplicación de tipos avanzados	14
• Justificación técnica de tipos avanzados	14
• JSONB	14
• TSTZRANGE	14
• Extensiones a utilizar con justificación.	15
• Justificación de extensiones	15
◦ pg_trgm	15
• PostGIS	15
• Estrategia de particionamiento	15
• Particiones mensuales:	15
• Justificación del particionamiento	16
Diseño lógico preliminar - Módulo MongoDB	16
• Colecciones principales identificadas	16

• Esquema de documentos - Estructura Simplificada	16
• 1. Colección <code>products_catalog</code>	16
• 2. Colección <code>reviews</code>	17
• 3. Colección <code>user_sessions</code>	18
• 4. Colección <code>analytics_events</code>	19
• Patrones de modelado a aplicar	19
1. Attribute Pattern - Patrón de Atributos:	19
2. Subset Pattern - Patrón de Subconjunto:	19
3. Extended Reference Pattern - Referencia Extendida:	19
4. Patrón de Expiración - TTL - Time To Live:	19
5. Time-Series Collections:	19
Decisiones arquitectónicas justificadas	20
• Matriz de decisión por entidad	20
• Análisis en profundidad del Teorema CAP y PACELC	21
Figura 3. Análisis de Aplicación del Teorema CAP en el Diseño Híbrido Propuesto para el Dataset Ollist.	21
1. PostgreSQL: Enfoque CP y PC en(PACELC)	21
2. MongoDB: Enfoque AP y PA/EL en (PACELC)	21
• Evaluación de Trade-offs y Estrategias de Mitigación	22
• Flujos de sincronización (PostgreSQL-MongoDB)	22
Anexos	23
• Diccionario de datos	23
• Scripts SQL preliminares	24
◦ Creación de extensiones	24
• Índices	24
• Trigger <code>updated_at</code>	25
• Ejemplos de datos	25
• Seed geolocation	25
• Seed customers	26
• Seed sellers	26
• Seed products	27
• Seed orders	27
• Seed order items	27
• Seed payments	28
• Seed promotions	28

Enfoque Arquitectónico Seleccionado

El equipo decide seleccionar la Arquitectura Políglota Híbrida, Opción 3 planteada en la guía previa. La solución propuesta es entonces híbrida basada en el Teorema CAP, buscando un balance entre consistencia y disponibilidad. Justificación de la Arquitectura:

- PostgreSQL: Principal almacenamiento para transacciones donde la integridad está en primer plano, como pagos y estados de envío.
- MongoDB: Elegido Almacenamiento de documentos para el catálogo, permitiendo que diferentes categorías de productos tengan atributos variables sin modificar el esquema global.

Abstract

Este documento presenta el diseño de la arquitectura híbrida de bases de datos para Ecommify, orientada a soportar cargas transaccionales (OLTP) y analíticas (OLAP). Mediante un enfoque de persistencia políglota, se distribuye el dominio de datos entre PostgreSQL y MongoDB para equilibrar consistencia, escalabilidad y disponibilidad. Las decisiones arquitectónicas clave incluyen:

- Núcleo Transaccional - PostgreSQL: Las entidades de Orders, Payments y Customers se asignaron a este motor relacional. Esta decisión garantiza el cumplimiento de propiedades ACID, integridad referencial y prevención de condiciones de carrera mediante bloqueos a nivel de fila durante operaciones críticas.
- Catálogo y Analítica - MongoDB: El Products Catalog, Reviews y Sessions se delegaron a este motor NoSQL. Su estructura polimórfica resuelve la heterogeneidad de los productos y optimiza la latencia en escenarios con un ratio de lectura/escritura superior a 100:1. Adicionalmente, facilita la retención de datos analíticos mediante índices TTL y soporte para series de tiempo.

La segregación estratégica garantiza que el sistema escale horizontalmente en la capa de consultas, manteniendo al mismo tiempo una consistencia inquebrantable en las operaciones financieras del negocio.

Análisis de Requisitos

- **Contexto de Ecomimfy**

Ecommify es una plataforma de comercio electrónico híbrida orientada a soportar operaciones transaccionales de alta concurrencia y procesamiento analítico de datos derivados de comportamiento de usuarios, catálogo de productos y trazabilidad logística.

La plataforma debe soportar:

- Gestión de clientes.
- Administración de pedidos.
- Procesamiento de pagos.
- Gestión de inventario.
- Catálogo dinámico de productos.
- Sistema de reseñas.
- Consultas analíticas y dashboards operacionales.

Debido al comportamiento heterogéneo de los datos y los diferentes patrones de acceso, se propone una arquitectura políglota que combina PostgreSQL y MongoDB, aprovechando las fortalezas de ambos motores.

- **Requisitos funcionales**

ID	Requisito
RF-01	Registrar clientes y autenticarlos en la plataforma
RF-02	Gestionar catálogo de productos con atributos dinámicos
RF-03	Permitir creación y actualización de pedidos
RF-04	Procesar pagos asociados a pedidos
RF-05	Gestionar inventario por vendedor
RF-06	Permitir reseñas y calificaciones de productos
RF-07	Consultar historial de compras
RF-08	Generar reportes y métricas comerciales
RF-09	Permitir búsquedas tolerantes a errores tipográficos
RF-10	Gestionar promociones y periodos de descuento

- **Requisitos no funcionales**

Categoría	Requisito
Escalabilidad	Soportar crecimiento horizontal del catálogo y consultas concurrentes
Consistencia	Garantizar integridad ACID en pedidos y pagos
Disponibilidad	Mantener acceso continuo al catálogo y reseñas
Rendimiento	Optimizar consultas OLTP y OLAP
Flexibilidad	Soportar atributos dinámicos de productos
Seguridad	Garantizar integridad referencial y control de acceso
Observabilidad	Permitir monitoreo de transacciones y consultas
Mantenibilidad	Facilitar evolución del esquema y nuevas funcionalidades

- **Identificación de entidades del dominio**

- PostgreSQL - Transaccional

Entidad	Descripción
customers	Información de clientes
orders	Pedidos realizados
order_items	Productos asociados a pedidos
payments	Información de pagos
promotions	Promociones temporales asociadas a productos
sellers	Vendedores
products	Tabla maestra mínima de productos usada como ancla relacional mediante product_id y sku.
geolocation	Información geográfica

- MongoDB - Documental

Colección	Descripción
products_catalog	Catálogo flexible de productos
reviews	Reseñas y comentarios
user_sessions	Sesiones y comportamiento
analytics_events	Eventos de navegación

- **Tabla de volúmenes de datos esperados**

Entidad	Volumen estimado	Crecimiento esperado	Tipo de carga
orders	10M+ registros	Alto	OLTP
order_items	50M+ registros	Alto	OLTP
payments	10M+ registros	Alto	OLTP
customers	2M+ registros	Medio	OLTP
products_catalog	500K+ documentos	Alto	Lectura intensiva
reviews	20M+ documentos	Alto	Escritura/lectura
analytics_events	500M+ eventos	Muy alto	OLAP/Eventos

- **Justificación arquitectónica**

La segmentación tecnológica responde a diferentes patrones de acceso y atributos de calidad.

PostgreSQL se utiliza en módulos de transacción y con una relación clara debido a:

- Soporte ACID,
- Integridad referencial,
- Manejo robusto de transacciones,
- Particionamiento,
- Extensiones avanzadas,
- Consultas complejas mediante SQL.

MongoDB se adopta para componentes con:

- Esquemas dinámicos,
- Alta variabilidad estructural,
- Consultas rápidas de lectura,
- Necesidad de escalabilidad horizontal.

Esta aproximación sigue un modelo de persistencia políglota alineado con la arquitectura propuesta para el e-commerce.

Diseño Conceptual

- **Diagrama ER completo de alta calidad - Sin Normalizar**

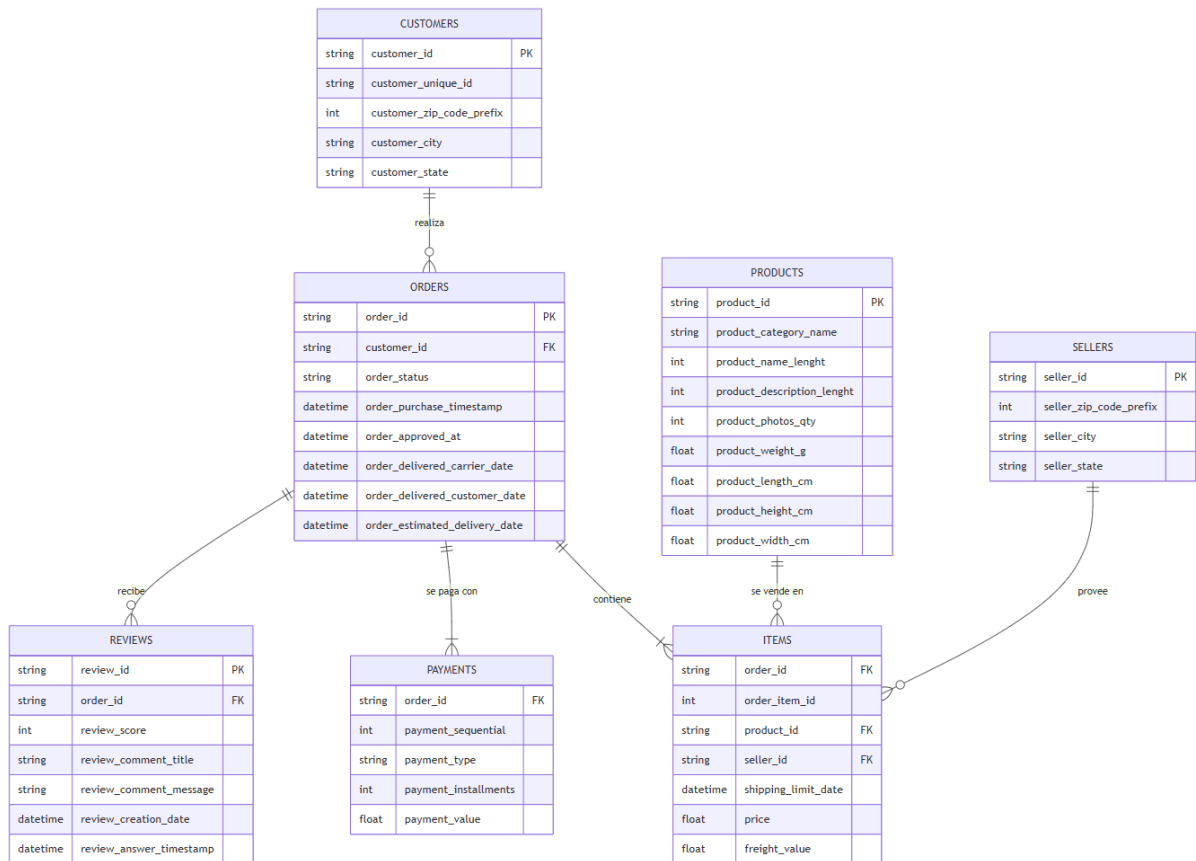


Figura 1. Diagrama ER Sin Normalizar del Dataset Principal Ollist.

- **Descripción detallada de cada entidad y relación**

- **Customers:** Representa a los compradores. Contiene el `customer_id`, para la transacción y el `customer_unique_id`, para seguimiento histórico del usuario.
- **Orders:** Registra el estado del ciclo de vida de la compra, creado, entregado, etc. Los tiempos de procesamiento.
- **Items:** Entidad asociativa, es el detalle del pedido. Desglosa cada producto dentro de una orden, incluyendo su precio y costo de envío.
- **Products:** Tabla maestra mínima de productos dentro del módulo relacional. No almacena atributos descriptivos, categorías, imágenes ni especificaciones, ya que esos datos se gestionan en MongoDB mediante la colección `products_catalog`. En PostgreSQL, `products` actúa como ancla relacional para garantizar que los ítems de pedido y las promociones referencian productos existentes.
- **Sellers:** Datos de los vendedores que ofrecen sus productos en la plataforma, permitiendo la gestión de un modelo multivendedor.
- **Payments:** Registra la dimensión financiera. Permite múltiples transacciones o métodos de pago para una sola orden.

- **Reviews:** Almacena la experiencia del usuario. Vincula la satisfacción del cliente directamente con un pedido específico.
- **Promotions:** Representa campañas comerciales temporales asociadas a productos. Permite registrar el nombre de la promoción, el porcentaje de descuento y su periodo de vigencia mediante el tipo avanzado `TSTZRANGE`. Su ubicación en PostgreSQL se justifica porque las promociones afectan reglas transaccionales, precios finales y validaciones de negocio durante el proceso de compra.

- **Descripción de Relaciones**

- **Customers (1) — (N) Orders:** un cliente puede realizar múltiples pedidos, pero cada pedido pertenece a un único cliente.
- **Orders (1) — (N) Order_Items:** un pedido puede contener uno o varios productos.
- **Orders (1) — (N) Payments:** un pedido puede pagarse mediante uno o varios registros de pago.
- **Products (1) — (N) Order_Items:** un producto puede aparecer en múltiples pedidos.
- **Sellers (1) — (N) Order_Items:** un vendedor puede participar en múltiples ventas.
- **Products (1) — (N) Promotions:** un producto puede tener múltiples promociones en diferentes periodos de tiempo.
- **Geolocation (1) — (N) Customers:** una ubicación puede estar asociada a varios clientes.
- **Geolocation (1) — (N) Sellers:** una ubicación puede estar asociada a varios vendedores.
- **Products PostgreSQL (1) — (1/N) Products_Catalog MongoDB:** la tabla products conserva la identidad transaccional del producto, mientras que MongoDB almacena su información enriquecida de catálogo.

- **Restricciones de negocio identificadas**

- Integridad Transaccional - Pedidos y Pagos
 - Sincronía de Pagos: El valor total registrado en `order_payments` debe coincidir exactamente con la suma de `price + freight_value` en `order_items`. No puede haber una orden pagada por un monto distinto al de sus productos.
 - Métodos de Pago: Solo se permiten tipos de pago predefinidos, `boleto`, `credit_card`, `voucher`, `debit_card`. Cualquier otro debe ser rechazado por el sistema.
 - Secuencia de Tiempo: La fecha de entrega `order_delivered_customer_date` no puede ser anterior a la fecha de compra `order_purchase_timestamp`.
 - Integridad de producto: todo `order_item` y toda `promotion` deben referenciar un `product_id` existente en la tabla `products`. Esto evita pedidos o promociones sobre productos inexistentes.
 - Vigencia promocional: toda promoción debe tener un `promotion_period` válido mediante `TSTZRANGE`, permitiendo consultar promociones activas, vencidas o futuras.
 - Porcentaje de descuento: `discount_percentage` debe estar entre 0 y 100 para evitar descuentos inválidos.

- Respuesta de pasarela: `gateway_response` se almacena en formato JSONB para conservar información técnica variable de la pasarela de pagos sin alterar el esquema relacional.
- Logística y Vendedores
 - Geolocalización Obligatoria: No se debe permitir el registro de un customer o seller cuyo `zip_code_prefix` no exista previamente en la tabla de geolocation. Esto asegura que siempre se pueda calcular el flete.
 - Límite de Envío: La fecha `shipping_limit_date` en los ítems es una restricción a considerar, si el vendedor no entrega al transportista antes de esa fecha, el pedido debe marcarse con una alerta de retraso.
- Calidad y Experiencia - Reviews
 - Reseña Post-Venta: Una review solo puede ser vinculada a una orden que ya tenga un estado de `delivered-entregado` o `shipped-enviado`, evitando reseñas falsas de productos no recibidos.
 - Rango de Calificación: El `review_score` debe estar estrictamente en el rango de 1 a 5.

Diseño lógico - Módulo PostgreSQL

- **Esquema relacional normalizado**

El modelo relacional es diseñado considerando el dataset original y siguiendo principios de normalización hasta Tercera Forma Normal (3FN), eliminando redundancias y garantizando integridad referencial.

Las entidades transaccionales principales son:

- geolocation
- customers
- sellers
- products
- orders
- order_items
- payments
- promotions

- Tabla geolocation

```
CREATE TABLE geolocation (  
  geolocation_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  zip_code VARCHAR(15) NOT NULL,  
  city VARCHAR(100) NOT NULL,  
  state CHAR(2) NOT NULL,  
  coordinates GEOGRAPHY(POINT, 4326),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

- Tabla customers

```
CREATE TABLE customers (  
    customer_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    customer_unique_id UUID NOT NULL,  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL,  
    phone VARCHAR(20),  
    geolocation_id UUID REFERENCES geolocation(geolocation_id),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT unique_customer_email UNIQUE (customer_unique_id,  
    email)  
);
```

- Tabla sellers

```
CREATE TABLE sellers (  
    seller_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    seller_name VARCHAR(150) NOT NULL,  
    email VARCHAR(150) UNIQUE,  
    geolocation_id UUID REFERENCES geolocation(geolocation_id),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

- Tabla products

```
CREATE TABLE products (  
    product_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    sku VARCHAR(50) UNIQUE NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

- Tabla orders

```
CREATE TABLE orders (  
    order_id UUID NOT NULL DEFAULT gen_random_uuid(),  
    customer_id UUID NOT NULL REFERENCES customers(customer_id),  
    order_status VARCHAR(30) NOT NULL DEFAULT 'CREATED',  
    purchase_timestamp TIMESTAMP NOT NULL,  
    delivered_timestamp TIMESTAMP,  
    total_amount NUMERIC(12,2) NOT NULL CHECK (total_amount >= 0),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (order_id, purchase_timestamp)  
) PARTITION BY RANGE (purchase_timestamp);
```

- Tabla order_items

```
CREATE TABLE order_items (  
  order_item_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  order_id UUID NOT NULL,  
  purchase_timestamp TIMESTAMP NOT NULL,  
  product_id UUID NOT NULL REFERENCES products(product_id),  
  seller_id UUID NOT NULL REFERENCES sellers(seller_id),  
  quantity INTEGER NOT NULL CHECK (quantity > 0),  
  price NUMERIC(10,2) NOT NULL CHECK (price >= 0),  
  freight_value NUMERIC(10,2) DEFAULT 0 CHECK (freight_value >= 0),  
  shipping_limit_date TIMESTAMP,  
  FOREIGN KEY (order_id, purchase_timestamp) REFERENCES  
  orders(order_id, purchase_timestamp)  
);
```

- Tabla payments

```
CREATE TABLE payments (  
  payment_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  order_id UUID NOT NULL,  
  purchase_timestamp TIMESTAMP NOT NULL,  
  payment_type VARCHAR(30) NOT NULL,  
  payment_installments INTEGER DEFAULT 1 CHECK  
  (payment_installments > 0),  
  payment_value NUMERIC(12,2) NOT NULL CHECK (payment_value >= 0),  
  gateway_response JSONB,  
  payment_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (order_id, purchase_timestamp) REFERENCES  
  orders(order_id, purchase_timestamp)  
);
```

- Tabla promotions

```
CREATE TABLE promotions (  
  promotion_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  product_id UUID NOT NULL REFERENCES products(product_id),  
  promotion_name VARCHAR(150) NOT NULL,  
  discount_percentage NUMERIC(5,2) CHECK (discount_percentage  
  BETWEEN 0 AND 100),  
  promotion_period TSTZRANGE NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

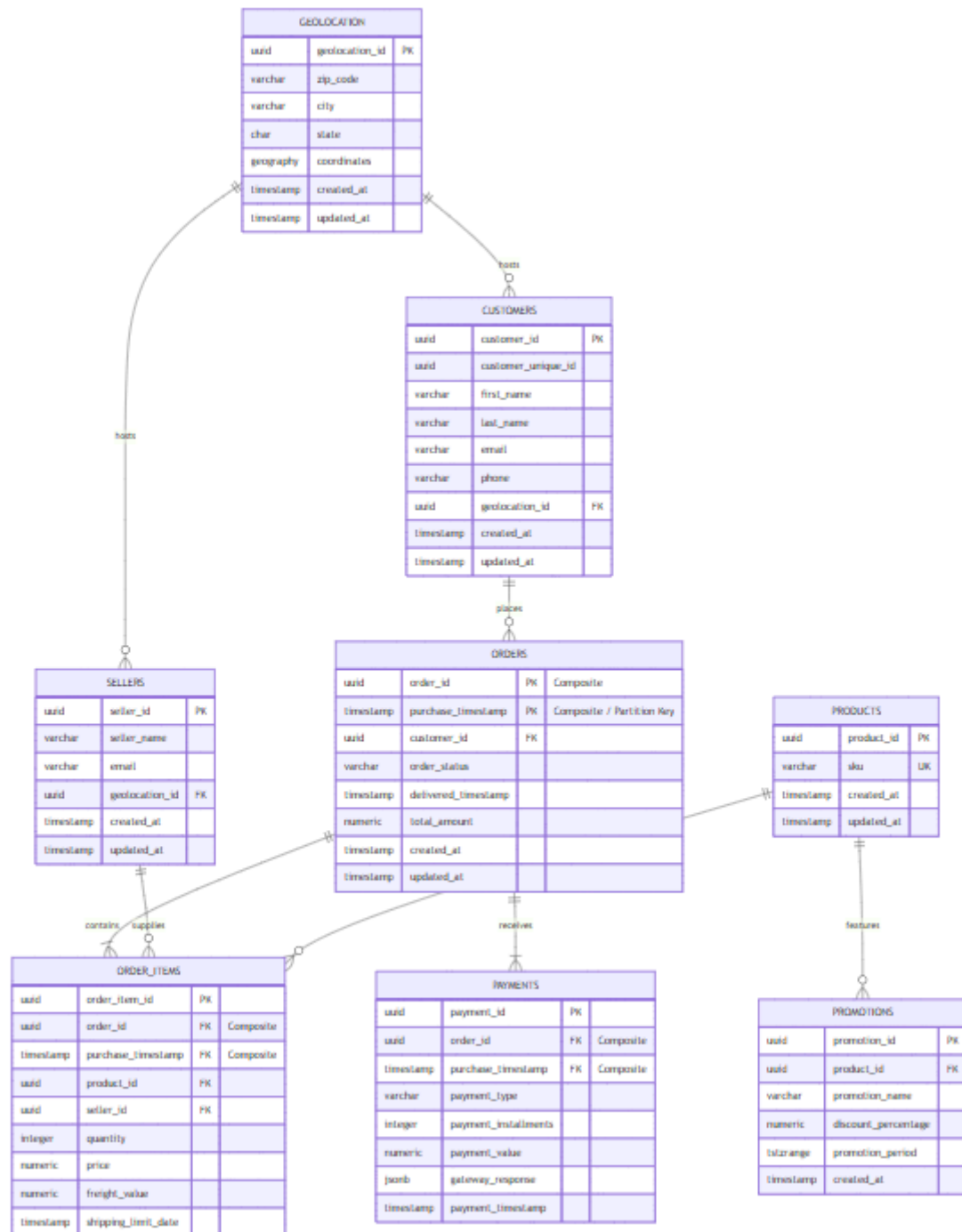


Figura 2. Diagrama ER Normalizado (3FN) para el Diseño Propuesto del Módulo PostgreSQL.

- **Aplicación de tipos avanzados**

Campo	Tipo avanzado	Justificación
gateway_response	JSONB	Respuesta de la pasarela de pagos
promotion_period	TSTZRANGE	Gestión temporal de promociones
coordinates	POINT/PostGIS	Consultas geoespaciales

- **Justificación técnica de tipos avanzados**

- **JSONB**

Se utiliza en la tabla payments, específicamente en el campo gateway_response, para almacenar la respuesta flexible de la pasarela de pagos. Esta respuesta puede variar según el proveedor e incluir campos como provider, status, authorization_code, transaction_id y response_message. Su uso evita alterar el esquema relacional ante cambios en la estructura técnica del gateway, manteniendo al mismo tiempo el pago dentro del núcleo transaccional de PostgreSQL.

Ejemplo:

gateway_response JSONB

Ejemplo de contenido:

```
{
  "provider": "Stripe",
  "status": "APPROVED",
  "authorization_code": "AUTH-20260115-001",
  "transaction_id": "TXN-ECOMMIFY-001",
  "response_message": "Payment approved successfully"
}
```

- **TSTZRANGE**

Permite representar periodos promocionales con operadores temporales eficientes:

promotion_period TSTZRANGE

Facilita consultas como:

- promociones activas
- traslapes
- vigencias temporales

- **Extensiones a utilizar con justificación.**

Extensión	Uso
pg_trgm	Búsqueda tolerante a errores
PostGIS	Consultas geoespaciales
pgcrypto	Generación segura de UUID

- **Justificación de extensiones**

- **pg_trgm**

Permite búsquedas tolerantes a errores sobre campos textuales del módulo relacional. En el diseño actual se aplica sobre `seller_name`, permitiendo localizar vendedores aunque existan errores tipográficos. La búsqueda textual enriquecida del catálogo de productos se delega a MongoDB, debido a que allí reside la información descriptiva del producto.

```
SELECT *  
FROM sellers  
WHERE seller_name % 'Tech Seler Brasil';
```

Mejora experiencia de usuario en búsquedas e-commerce.

- **PostGIS**

Permite:

- Cálculo de distancias,
- Optimización logística,
- Análisis geográfico de ventas.

Ejemplo:

```
ST_Distance()
```

- **Estrategia de particionamiento**

La tabla orders será particionada por rango temporal:

PARTITION BY RANGE (purchase_timestamp)

- **Particiones mensuales:**

- orders_2026_01
- orders_2026_02
- orders_2026_03
- orders_2026_04

- **Justificación del particionamiento**

Debido al alto volumen transaccional identificado en el dataset Olist, el particionamiento:

- Reduce tiempos de consulta,
- Mejora mantenimiento,
- Optimiza VACUUM,
- Facilita archivado histórico,
- Disminuye el impacto de índices masivos.
- Además, soporta estrategias híbridas hot/cold data.

Diseño lógico preliminar - Módulo MongoDB

En el marco de nuestra arquitectura híbrida, MongoDB actuará como el motor de base de datos principal para las operaciones de lectura intensiva Heavy-Read, y el manejo de datos con esquemas variables.

- **Colecciones principales identificadas**

Como se menciona en el diseño conceptual, el ecosistema NoSQL Documental gestionará las siguientes colecciones:

Colección	Descripción
products_catalog	Catálogo flexible de productos
reviews	Reseñas y comentarios
user_sessions	Sesiones y comportamiento
analytics_events	Eventos de navegación

- **Esquema de documentos - Estructura Simplificada**

A continuación, se presentan los esquemas para todas las colecciones de MongoDB, mostrando únicamente la estructura clave, los tipos de datos y los campos requeridos.

- 1. Colección products_catalog:

Este documento consolida la información del producto para evitar múltiples JOINS al momento de renderizar la página de detalles.

```
{
  "type": "object",
  "properties": {
    "_id": { "type": "string" },
    "product_id_pg": { "type": "string" },
    "category_name": { "type": "string" },
    "name": { "type": "string" },
    "description": { "type": "string" },
    "images": {
      "type": "array",
      "items": { "type": "string" }
    },
    "attributes": {
      "type": "array",

```

```

    "items": {
      "type": "object",
      "properties": {
        "k": { "type": "string" },
        "v": { "type": "string" }
      }
    },
    "display_price": { "type": "number" },
    "seller": {
      "type": "object",
      "properties": {
        "seller_id_pg": { "type": "string" },
        "name": { "type": "string" },
        "reputation_score": { "type": "number" }
      }
    },
    "top_reviews": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "score": { "type": "number" },
          "comment": { "type": "string" },
          "user_name": { "type": "string" }
        }
      }
    },
    "created_at": { "type": "string" },
    "updated_at": { "type": "string" }
  },
  "required": [
    "product_id_pg",
    "name",
    "display_price",
    "seller"
  ]
}

```

- 2. Colección reviews:

Documento que almacena el histórico completo de reseñas de forma desnormalizada.

```

{
  "type": "object",
  "properties": {
    "_id": { "type": "string" },
    "order_id_pg": { "type": "string" },
    "product_id_pg": { "type": "string" },
    "customer": {
      "type": "object",
      "properties": {

```

```

        "customer_id_pg": { "type": "string" },
        "first_name": { "type": "string" }
    }
},
"score": { "type": "integer" },
"title": { "type": "string" },
"message": { "type": "string" },
"creation_date": { "type": "string" }
},
"required": [
    "order_id_pg",
    "product_id_pg",
    "customer",
    "score",
    "creation_date"
]
}

```

- 3. Colección `user_sessions`:

Documento diseñado para rastrear sesiones activas y carritos de compra abandonados.

```

{
  "type": "object",
  "properties": {
    "_id": { "type": "string" },
    "customer_id_pg": { "type": "string" },
    "ip_address": { "type": "string" },
    "device_info": { "type": "string" },
    "cart_items": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "product_id_pg": { "type": "string" },
          "quantity": { "type": "integer" },
          "price_at_addition": { "type": "number" }
        }
      }
    },
    "created_at": { "type": "string" },
    "last_activity": { "type": "string" }
  },
  "required": [
    "ip_address",
    "created_at",
    "last_activity"
  ]
}

```

- 4. Colección `analytics_events`:

Estructura orientada a series de tiempo Time-Series para rastrear flujos de clics y vistas.

```
{
  "type": "object",
  "properties": {
    "_id": { "type": "string" },
    "session_id": { "type": "string" },
    "event_type": { "type": "string" },
    "product_id_pg": { "type": "string" },
    "page_url": { "type": "string" },
    "event_metadata": { "type": "object" },
    "timestamp": { "type": "string" }
  },
  "required": [
    "session_id",
    "event_type",
    "timestamp"
  ]
}
```

- **Patrones de modelado a aplicar**

1. Attribute Pattern - Patrón de Atributos:

Utilizado en la colección `products_catalog` en el campo `attributes`. En lugar de tener un esquema rígido por cada tipo de producto, estandarizamos un array de clave-valor (k y v). Esto permite indexar el campo `attributes.k` y `attributes.v` para realizar filtros dinámicos de forma altamente eficiente.

2. Subset Pattern - Patrón de Subconjunto:

Implementado al incrustar el arreglo `top_reviews` dentro del documento del producto. Al cargar la página del producto, el usuario verá inmediatamente las reseñas destacadas sin necesidad de hacer una consulta adicional a la colección `reviews`.

3. Extended Reference Pattern - Referencia Extendida:

En lugar de guardar solo el `seller_id_pg`, guardamos también el nombre y la reputación del vendedor dentro del documento del producto para evitar múltiples saltos de consulta.

4. Patrón de Expiración - TTL - Time To Live:

Se aplica un índice TTL en el campo `last_activity` de la colección `user_sessions`. Si un usuario abandona su carrito y no hay actividad en 7 días, MongoDB eliminará el documento automáticamente, liberando espacio de forma autónoma.

5. Time-Series Collections:

Para la colección `analytics_events`, se utilizará la funcionalidad nativa de Time-Series de MongoDB, la cual optimiza el almacenamiento en disco y mejora el rendimiento de consultas agregadas sobre rangos de tiempo, ideal para paneles de Business Intelligence por ejemplo.

Decisiones arquitectónicas justificadas

- **Matriz de decisión por entidad**

Entidad / Dominio	Motor Seleccionado	Carga / Patrón	Justificación Técnica
Orders	PostgreSQL	OLTP (Write-Heavy)	Registro del ciclo de vida de la compra. Requiere aislamiento y particionamiento por rango <code>purchase_timestamp</code> para optimizar el rendimiento histórico sin degradar transacciones activas.
Payments	PostgreSQL	OLTP (Write-Heavy)	Dimensión financiera que soporta múltiples transacciones y cuotas por orden. Exige atomicidad absoluta para mitigar riesgos de fraude o inconsistencias en pasarelas de pago.
Customers / Sellers	PostgreSQL	OLTP (Read/Write Mixto)	Gestión de actores del ecosistema multivendedor. Requiere normalización (3FN) conectada a geolocation para auditoría logística, e integridad compuesta para tracking histórico.
Products	PostgreSQL	OLTP (Read-Heavy)	Tabla maestra simple cuyo único propósito es servir de ancla relacional e impedir que se procesen pedidos de ítems inexistentes.
Promotions	PostgreSQL	OLTP / Reglas comerciales temporales	Requiere integridad referencial con productos, control temporal mediante TSTZRANGE y consistencia para evitar aplicación de descuentos inválidos.
Products Catalog	MongoDB	OLAP/Lecturas	Estructura polimórfica. Beneficia la latencia baja en consultas por índice de atributos y búsqueda de texto. Ratio Lectura/Escritura > 100:1.
Reviews	MongoDB	Documental / Lectura	Datos desestructurados. Alta escalabilidad horizontal. Integración sencilla con pipelines de agregación.
Analytics / Sessions	MongoDB	Write-Heavy (Series)	Soporte natural para documentos Time-Series y eliminación automática mediante índices TTL.

- **Análisis en profundidad del Teorema CAP y PACELC**

El Teorema CAP establece que, en un sistema distribuido propenso a particiones de red (P), es imposible garantizar simultáneamente la Consistencia (C) estricta y la Disponibilidad (A).

Adicionalmente, el teorema PACELC expande esto indicando que, incluso cuando no hay particiones (E - Else), se debe elegir entre Latencia (L) y Consistencia (C).

La arquitectura híbrida aborda estos teoremas separando los dominios de datos para satisfacer diferentes necesidades del negocio:

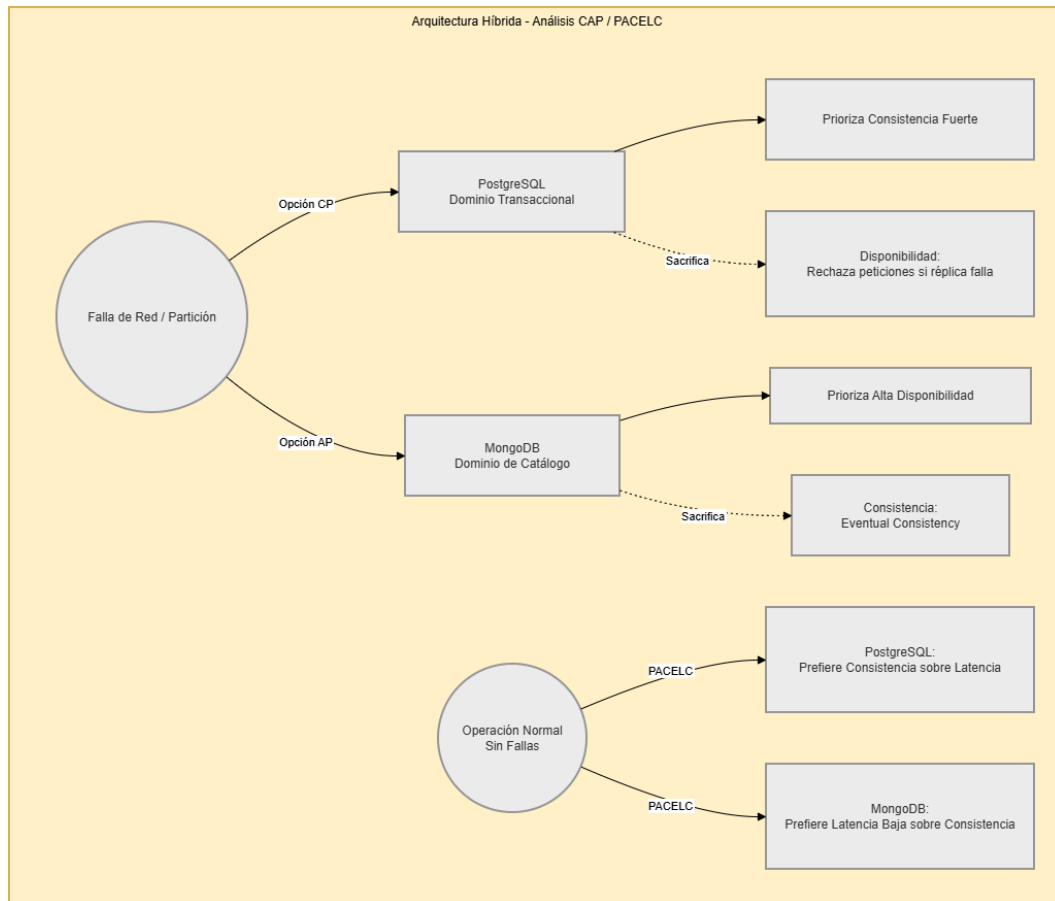


Figura 3. Análisis de Aplicación del Teorema CAP en el Diseño Híbrido Propuesto para el Dataset Ollist.

1. PostgreSQL: Enfoque CP y PC en(PACELC)

- Comportamiento ante particiones (P): Si el nodo primario pierde conexión con las réplicas síncronas necesarias para alcanzar el quórum, el clúster bloqueará nuevas escrituras (rechaza pagos/órdenes) para evitar estados divergentes. (Elige C, sacrifica A).
- Comportamiento normal (E): Se prefiere la Consistencia (C) sobre la Latencia (L). Cada escritura pasa por un proceso estricto de commit en el disco (WAL) antes de responder al cliente, garantizando ACID, aunque la latencia de escritura sea mayor.

2. MongoDB: Enfoque AP y PA/EL en (PACELC)

- Comportamiento ante particiones (P): Si un nodo secundario de MongoDB pierde conexión con el primario, continuará sirviendo lecturas de catálogo utilizando su data local, aunque esté potencialmente obsoleta. El usuario sigue navegando y comprando sin percibir caídas. (Elige A, sacrifica C).
- Comportamiento normal (E): Se prefiere la Baja Latencia (L) sobre la Consistencia (C). Se utilizan Secondary Reads para balancear la carga globalmente, permitiendo latencias de lectura <10ms, aceptando que un usuario podría ver un precio que cambió hace milisegundos.

- **Evaluación de Trade-offs y Estrategias de Mitigación**

La persistencia políglota introduce beneficios innegables para la escala, pero requiere gobernar la complejidad técnica. A continuación, detallamos los compromisos asumidos:

Decisión Arquitectónica (Trade-off)	Beneficio Esperado	Riesgo o Desventaja	Estrategia de Mitigación Detallada
<i>Separar PG y Mongo</i>	Soporta un ratio >100:1 de lecturas/escrituras. Latencia P95 < 50ms en catálogo.	Venta de productos agotados por desfase.	Soft Allocations & UX: Validar stock real solo al añadir al carrito o iniciar checkout. Implementar reservas temporales en Redis (ej. 5 mins) durante el pago. Mostrar mensajes UX como "Últimas unidades".
Desnormalización en MongoDB	Latencia ultrabaja. Se evitan JOINS y peticiones múltiples para renderizar una página de producto.	Si un vendedor cambia su nombre en PG, miles de productos en Mongo podrían mostrar el nombre antiguo.	PG es la única fuente de verdad. Se implementa un Flujo de Sincronización Estricto (CDC). Puesto que no se editan datos maestros directamente en MongoDB.
Consistencia Eventual Asíncrona	Bajo acoplamiento. PG no espera a Mongo para confirmar un guardado, mejorando la latencia de escritura.	Si el servicio de sincronización falla o la red cae, MongoDB queda desincronizado permanentemente.	Uso de una tabla <code>outbox_events</code> en la misma transacción de PG y una cola (Kafka/RabbitMQ) para garantizar At-Least-Once Delivery (entrega al menos una vez).
Operación de Dos Motores Diferentes	Uso de la herramienta adecuada para el trabajo adecuado (SQL para finanzas, NoSQL para catálogos).	Duplicación de esfuerzos en monitoreo, backups, despliegues y curva de aprendizaje del equipo.	Uso obligatorio de servicios gestionados (ej. Supabase para PG, Atlas para Mongo). Uso de infraestructura como Código (Terraform) y observabilidad centralizada.

- **Flujos de sincronización (PostgreSQL-MongoDB)**

Para mantener la cohesión de los datos sin crear un acoplamiento síncrono que degrade el rendimiento, es diseñada una arquitectura reactiva basada en la combinación del Patrón: Change Data Capture (CDC) con Patrón Outbox:

1. Origen y Transacción - PostgreSQL: Cuando se crea o actualiza un producto transaccional en PostgreSQL, la tabla `products` conserva únicamente su identidad mínima (`product_id`, `sku`). La información enriquecida del catálogo, como nombre, descripción, imágenes, categoría y atributos dinámicos, se gestiona en MongoDB mediante `products_catalog`. La sincronización se orienta a mantener correspondencia entre el identificador relacional del producto y su representación documental, usando eventos de integración para crear o actualizar el documento correspondiente en MongoDB.

2. Captura - Debezium: Debezium lee continuamente el Write-Ahead Log (WAL) de PostgreSQL, detectando los nuevos registros en la tabla outbox_events.
3. Distribución - Apache Kafka / EventBridge: El CDC emite el evento al tópico catalog.product.updated.
4. Consumo y Transformación - Worker Node: Un microservicio suscriptor recibe el evento, ejecuta la transformación necesaria (por ejemplo, adaptar el esquema rígido relacional al Attribute Pattern de Mongo) y ejecuta una operación de updateOne con la opción upsert: true en MongoDB.

Hipótesis: El catálogo en MongoDB se mantiene eventualmente consistente, en el orden de los milisegundos, aislando completamente el rendimiento de escritura transaccional de las pesadas consultas de lectura del portal de cara al cliente.

Anexos

- **Diccionario de datos**

Tabla	Campo	Tipo	Restricción	Descripción
customers	customer_unique_id	VARCHAR(50)	NOT NULL	Identificador externo/histórico del cliente
geolocation	coordinates	GEOGRAPHY(POINT,4326)	NULL	Coordenadas geográficas
products	sku	VARCHAR(50)	UNIQUE, NOT NULL	Código único del producto
payments	gateway_response	JSONB	NULL	Respuesta flexible de la pasarela de pagos
promotions	promotion_id	UUID	PK	Identificador de promoción
promotions	product_id	UUID	FK	Producto asociado
promotions	promotion_name	VARCHAR(150)	NOT NULL	Nombre de campaña
promotions	discount_percentage	NUMERIC(5,2)	CHECK 0-100	Porcentaje de descuento
promotions	promotion_period	TSTZRANGE	NOT NULL	Vigencia temporal de la promoción

- **Scripts SQL preliminares**

- Creación de extensiones

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS hstore;
```

- **Índices**

```
CREATE INDEX idx_customers_unique_id
    ON customers(customer_unique_id);
CREATE INDEX idx_customers_email
    ON customers(email);
CREATE INDEX idx_customers_geolocation
    ON customers(geolocation_id);
CREATE INDEX idx_sellers_email
    ON sellers(email);
CREATE INDEX idx_sellers_geolocation
    ON sellers(geolocation_id);
CREATE INDEX idx_products_sku
    ON products(sku);
CREATE INDEX idx_orders_customer_date
    ON orders(customer_id, purchase_timestamp);
CREATE INDEX idx_orders_status
    ON orders(order_status);
CREATE INDEX idx_orders_purchase_timestamp
    ON orders(purchase_timestamp);
CREATE INDEX idx_order_items_order
    ON order_items(order_id, purchase_timestamp);
CREATE INDEX idx_order_items_product
    ON order_items(product_id);
CREATE INDEX idx_order_items_seller
    ON order_items(seller_id);
CREATE INDEX idx_payments_order
    ON payments(order_id, purchase_timestamp);
CREATE INDEX idx_geolocation_coordinates
    ON geolocation USING GIST(coordinates);
CREATE INDEX idx_promotions_product
    ON promotions(product_id);
CREATE INDEX idx_promotions_period
    ON promotions USING GIST(promotion_period);
CREATE INDEX idx_payments_gateway_response_gin
    ON payments USING GIN(gateway_response);
CREATE INDEX idx_mv_sales_by_seller_monthly
    ON mv_sales_by_seller_monthly(seller_id,
    sales_month);
```

- **Trigger updated_at**

```
CREATE OR REPLACE FUNCTION update_timestamp()  
RETURNS TRIGGER AS $$  
BEGIN  
    NEW.updated_at = CURRENT_TIMESTAMP;  
RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_geolocation_updated_at  
BEFORE UPDATE ON geolocation  
FOR EACH ROW  
EXECUTE FUNCTION update_timestamp();
```

```
CREATE TRIGGER trg_customers_updated_at  
BEFORE UPDATE ON customers  
FOR EACH ROW  
EXECUTE FUNCTION update_timestamp();
```

```
CREATE TRIGGER trg_sellers_updated_at  
BEFORE UPDATE ON sellers  
FOR EACH ROW  
EXECUTE FUNCTION update_timestamp();
```

```
CREATE TRIGGER trg_products_updated_at  
BEFORE UPDATE ON products  
FOR EACH ROW  
EXECUTE FUNCTION update_timestamp();
```

```
CREATE TRIGGER trg_orders_updated_at  
BEFORE UPDATE ON orders  
FOR EACH ROW  
EXECUTE FUNCTION update_timestamp();
```

- **Ejemplos de datos**

- **Seed geolocation**

```
INSERT INTO geolocation (  
    geolocation_id,  
    zip_code,  
    city,  
    state,  
    coordinates  
)  
VALUES  
(  
    '11111111-1111-1111-1111-111111111111',  
    '01000',
```

```

        'Sao Paulo',
        'SP',
        ST_GeogFromText('POINT(-46.6333 -23.5505)')
    ),
    (
        '22222222-2222-2222-2222-222222222222',
        '20000',
        'Rio de Janeiro',
        'RJ',
        ST_GeogFromText('POINT(-43.1729 -22.9068)')
    );

```

- **Seed customers**

```

INSERT INTO customers (
    customer_id,
    customer_unique_id,
    first_name,
    last_name,
    email,
    phone,
    geolocation_id
)
VALUES
    (
        'aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaaa',
        '99999999-aaaa-aaaa-aaaa-aaaaaaaaaaaaa',
        'Pepito',
        'Perez',
        'pepito@email.com',
        '3001234567',
        '11111111-1111-1111-1111-111111111111'
    );

```

- **Seed sellers**

```

INSERT INTO sellers (
    seller_id,
    seller_name,
    email,
    geolocation_id
)
VALUES
    (
        'bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbbb',
        'Tech Seller Brasil',
        'seller@ecommify.com',
        '22222222-2222-2222-2222-222222222222'
    );

```

- **Seed products**

```
INSERT INTO products (  
    product_id,  
    sku  
)  
VALUES  
    (  
        'dddddddd-dddd-dddd-dddd-dddddddddddd',  
        'LAP-RYZ7-RTX4060-001'  
    );
```

- **Seed orders**

```
INSERT INTO orders (  
    order_id,  
    customer_id,  
    order_status,  
    purchase_timestamp,  
    total_amount  
)  
VALUES  
    (  
        'ffffffff-ffff-ffff-ffff-ffffffffffffff',  
        'aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaaa',  
        'APPROVED',  
        '2026-01-15 10:30:00',  
        1500000.00  
    );
```

- **Seed order items**

```
INSERT INTO order_items (  
    order_item_id,  
    order_id,  
    purchase_timestamp,  
    product_id,  
    seller_id,  
    quantity,  
    price,  
    freight_value,  
    shipping_limit_date  
)  
VALUES  
    (  
        '99999999-9999-9999-9999-999999999999',  
        'ffffffff-ffff-ffff-ffff-ffffffffffffff',  
        '2026-01-15 10:30:00',  
        'dddddddd-dddd-dddd-dddd-dddddddddddd',
```

```
'bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb',  
1,  
1500000.00,  
35000.00,  
'2026-01-20 23:59:59'  
);
```

- **Seed payments**

```
INSERT INTO payments (  
    payment_id,  
    order_id,  
    purchase_timestamp,  
    payment_type,  
    payment_installments,  
    payment_value,  
    gateway_response  
)  
VALUES  
(  
    '88888888-8888-8888-8888-888888888888',  
    'ffffffff-ffff-ffff-ffff-ffffffffffffff',  
    '2026-01-15 10:30:00',  
    'CREDIT_CARD',  
    3,  
    1500000.00,  
    '{  
        "provider": "Stripe",  
        "status": "APPROVED",  
        "authorization_code": "AUTH-20260115-001",  
        "transaction_id": "TXN-ECOMMIFY-001",  
        "response_message": "Payment approved successfully"  
    }'::jsonb  
)
```

- **Seed promotions**

```
INSERT INTO promotions (  
    promotion_id,  
    product_id,  
    promotion_name,  
    discount_percentage,  
    promotion_period  
)  
VALUES  
(  
    '77777777-7777-7777-7777-777777777777',  
    'ddddddddd-dddd-dddd-dddd-dddddddddddd',  
    'January Tech Promo',
```

```
10.00,  
TSTZRANGE('2026-01-01 00:00:00-05', '2026-02-01  
00:00:00-05')  
);
```